JAVA 2

Module FX - la calculatrice

Pas à pas, de la vue au modèle

Frédéric Ferrant – 2026-2027

Au programme

1. Le projet
2. La vue
3. Le contrôleur
4. Le modèle

1. Ce qu'on va construire



Une calculatrice simplifiée

Les quatre opérations, sur des entiers seulement

Les opérations s'enchaînent dans l'ordre où on les tape

2 x 3 + 4 x 5 donne donc 50, et non 26

Trois fichiers, trois rôles

La vue : `calculatrice.fxml`, à écrire en premier

Le contrôleur : `Controleur`, qui réagit aux boutons

Le modèle : `Modele`, qui calcule, sans rien savoir de `JavaFX`

Fournis : `ApplicationCalculatrice` et `Lanceur`

```
src/main/java/be/ephec/java/fx/e09/
    ApplicationCalculatrice.java   // fournie
    Lanceur.java                   // fournie
    Controleur.java                // a ecrire
    Modele.java                    // a ecrire
src/main/resources/be/ephec/java/fx/e09/
    calculatrice.fxml              // a ecrire
```

L'application charge la vue

`start` charge le fichier FXML, et en fait la scène

La racine du FXML nomme le contrôleur : `fx:controller`

`JavaFX` crée lui-même le contrôleur, et le relie à la vue

```
FXMLLoader chargeur = new FXMLLoader(
        ApplicationCalculatrice.class.getResource("calculatrice.fxml"));

stage.setScene(new Scene(chargeur.load()));
```

Les composants utilisés

Composant	Rôle dans la calculatrice
`VBox`	la racine : empile les lignes de haut en bas
`StackPane`	l'écran : superpose un cadre et un texte
`Rectangle`	le cadre de l'écran
`Text`	le nombre affiché
`HBox`	une ligne de boutons, de gauche à droite
`Button`	une touche

2. La vue, pas à pas

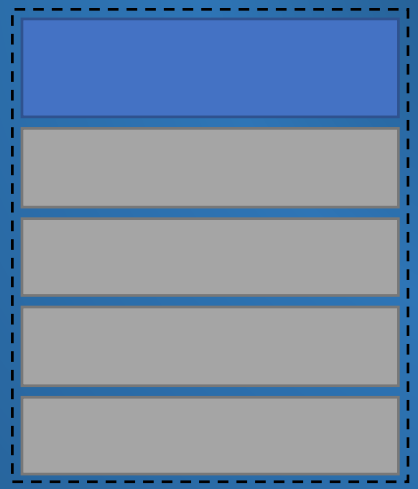


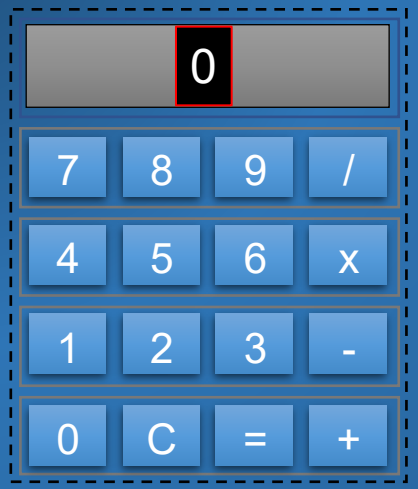
Le plan de l'écran

Une VBox de 300 sur 300, espacement 10

En haut, un StackPane : un cadre de 230 sur 50, et le texte

Puis quatre HBox de quatre boutons de 50 sur 50, espacés de 10





Deux mots de XML

Un élément s'ouvre et se ferme : `<HBox> ... </HBox>`, ou `<Text />`

Il peut contenir des attributs : `<Button text="7" />`

Les éléments s'emboîtent : l'emboîtement est l'arbre

```
<marin id="12">
    <nom>Surcouf</nom>
    <prenom>Robert</prenom>
</marin>
```

Le squelette

La racine est la `VBox`, qui nomme le contrôleur

Ses enfants : un `StackPane`, puis quatre `HBox`, vides pour l'instant

```
<VBox xmlns:fx="http://javafx.com/fxml"
      fx:controller="be.ephec.java.fx.e09.Controleur">
    <StackPane></StackPane>
    <HBox></HBox>
    <HBox></HBox>
    <HBox></HBox>
    <HBox></HBox>
</VBox>
```

Un attribut, c'est un setter

Chaque élément est une classe Java, chaque attribut une méthode `set`

`setAlignment` devient l'attribut `alignment`

La Javadoc de la classe dit donc quels attributs existent

```
<VBox prefWidth="300" prefHeight="300" spacing="10" alignment="CENTER" ...>
    <StackPane alignment="CENTER"></StackPane>
```

Le cadre de l'écran

Dans le `StackPane`, un `Rectangle` de 230 sur 50

Chaque classe utilisée doit être importée en tête du fichier

IntelliJ propose l'`import` manquant, comme pour le code Java

```
<?import javafx.scene.layout.*?>
<?import javafx.scene.shape.*?>
<?import javafx.scene.text.*?>
<?import javafx.scene.control.*?>

<StackPane alignment="CENTER">
    <Rectangle width="230" height="50" fill="TRANSPARENT" stroke="GRAY"/>
</StackPane>
```

Le texte, relié au contrôleur

Par-dessus le cadre, un `Text` qui affichera les nombres

`fx:id` lui donne un nom ; le contrôleur a un champ du même nom

`@FXML` dit à `JavaFX` d'y injecter l'objet créé

```
<StackPane alignment="CENTER">
    <Rectangle width="230" height="50" fill="TRANSPARENT" stroke="GRAY"/>
    <Text fx:id="affichage" text="0"/>
</StackPane>

public class Controleur {
    @FXML
    private Text affichage;
}
```

Une ligne de touches

Chaque `HBox` est espacée de 10, comme la `VBox`

Elle contient quatre `Button` de 50 sur 50, avec leur texte

```
<HBox spacing="10">
    <Button text="7" prefWidth="50" prefHeight="50"/>
    <Button text="8" prefWidth="50" prefHeight="50"/>
    <Button text="9" prefWidth="50" prefHeight="50"/>
    <Button text="/" prefWidth="50" prefHeight="50"/>
</HBox>
```

Un défaut à corriger

Dans *Scene Builder*, les lignes de touches sont collées à gauche

Le cadre, lui, est centré : rien n'est aligné

À vous de trouver l'attribut qui corrige cela

Vérifier, puis lancer

Scene Builder montre la vue à chaque étape, sans lancer

Lancer `Lanceur` affiche la calculatrice

Les boutons ne font rien encore : c'est normal

Un dernier détail, facultatif

`fx:define` crée un objet réutilisable dans la vue

Une police de 18, définie une fois, sert au texte et aux touches

```
<fx:define>
    <Font fx:id="POLICE" size="18"/>
</fx:define>

<Text fx:id="affichage" text="0" font="$POLICE"/>
<Button text="7" prefWidth="50" prefHeight="50" font="$POLICE"/>
```

3. Le contrôleur



Que doit faire chaque touche ?

Un comportement par touche ? Un seul pour toutes ?

Deux familles : les chiffres modifient le nombre affiché

Les opérateurs calculent, et retiennent ce qui a été tapé

Et « C » remet tout à zéro

Trois méthodes, branchées sur la vue

Une méthode du contrôleur, marquée `@FXML`, reçoit un `ActionEvent`

Dans le FXML, `onAction="#nom"` désigne la méthode à appeler

Pour commencer, chaque méthode écrit un mot dans la console

```
@FXML
private void chiffrePresse(ActionEvent evenement) {
    System.out.println("chiffre");
}

<Button text="7" prefWidth="50" prefHeight="50" onAction="#chiffrePresse"/>
<Button text="+" prefWidth="50" prefHeight="50" onAction="#operateurPresse"/>
```

Quelle touche a été pressée ?

Dix boutons, une seule méthode : c'est l'événement qui le dit

`getSource` retourne le bouton cliqué, `getText` son texte

```
String chiffre = ((Button) evenement.getSource()).getText();
```

Les chiffres

Si l'écran montre « 0 », le chiffre le remplace

Sinon, il s'accole à la fin : 56 puis 2 donne 562

`affichage.getText()` lit l'écran, `affichage.setText(...)` l'écrit

À vous d'écrire `chiffrePresse`

Et les opérateurs ?

Un opérateur ne modifie pas seulement l'écran : il calcule

Avant de l'écrire, il faut savoir qui calcule

Comment programmeriez-vous une calculatrice sans fenêtre ?

4. Le modèle, et la fin



Ce que fait le modèle

Une seule méthode : `calculer(premier, second, operateur)`

Elle choisit l'opération d'après l'opérateur, et retourne le résultat

Une division par zéro lève une `ArithmeticException`

À vous d'écrire `Modele`, puis sa classe de test

```
public long calculer(long premier, long second, String operateur) {
    return switch (operateur) {
        case "+" -> premier + second;
        // ... les trois autres, et la division par zero
    };
}
```

Le contrôleur connaît le modèle

Le contrôleur crée son modèle, et le garde dans un champ

Il retient aussi, entre deux clics, ce qu'on a tapé

```
private final Modele modele = new Modele();

private long premierNombre = 0;
private String operateurEnAttente = "";
private boolean nouvelleSaisie = true;
```

Les chiffres, une seconde règle

Après un opérateur, le chiffre suivant repart de zéro

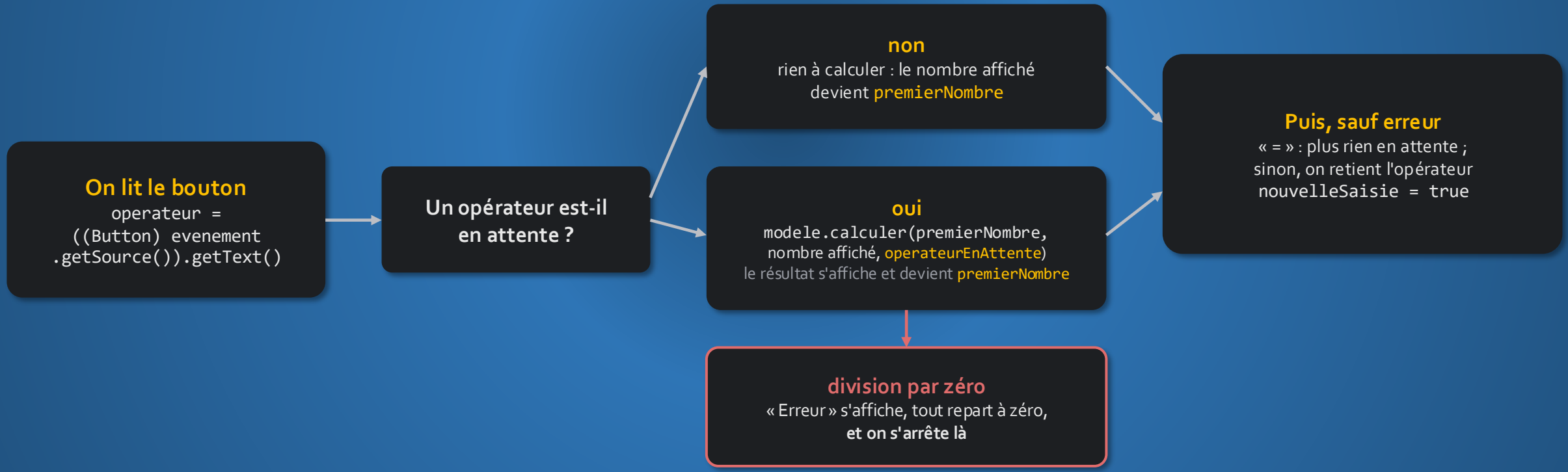
Sinon, 5 + 6 afficherait 56

C'est le rôle de `nouvelleSaisie`

À vous : les opérateurs

`operateurPresse` : l'arbre donne la structure, à vous le code

Et `effacer` remet l'écran à o, et oublie ce qui était en attente



Vérifier la calculatrice

12 + 5 = affiche 17 ; 13 / 4 = affiche 3

2 + 3 + 4 = affiche 9 : chaque opérateur calcule ce qui précède

1 / 0 = affiche « Erreur », puis la calculatrice repart

« C » remet tout à zéro, y compris l'opérateur en attente

Et après ?

Taper les chiffres au clavier

Respecter la priorité des opérations, puis les parenthèses

Ajouter des opérations : modulo, puissance

Exercices



FX.9 Une calculatrice, en MVC

fascicule FX